

A* (A-Star) Search Algorithm Using Python

Informed Search Algorithms — Artificial Intelligence Lab

Aim

To implement the A* (A-Star) Search Algorithm using Python to find the shortest path between a start node and a goal node in a weighted graph.

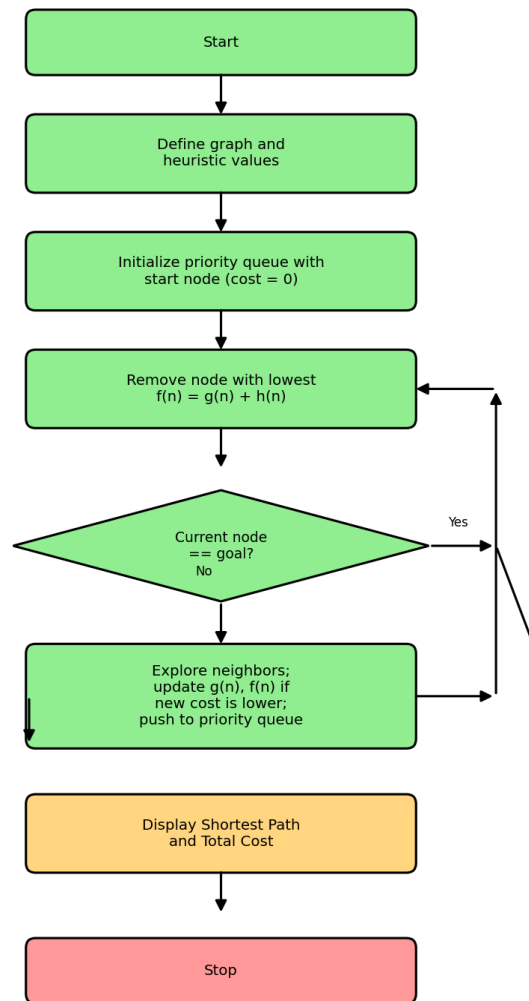
Objective

- To understand the working of the A* search algorithm.
- To find the shortest path from a source node to a destination node.
- To use heuristic values to improve search efficiency.
- To implement the A* algorithm using Python.
- To analyze the performance of informed search algorithms.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Initialize the priority queue with the start node.
4. Set the cost of the start node to 0.
5. Repeat until the priority queue is empty:
 - Remove the node with the lowest $f(n) = g(n) + h(n)$ value.
 - If the current node is the goal node, display the shortest path and stop.
 - Otherwise, explore all neighboring nodes.
 - Calculate the new cost for each neighbor.
 - If the new cost is lower than the previous cost, update the cost, calculate the new $f(n)$ value, and add the neighbor to the priority queue.
6. If no path exists, display "No Path Found."
7. Stop.

Flowchart



Code

```
from queue import PriorityQueue

graph = {
    'A': [( 'B', 1), ( 'C', 3)],
    'B': [( 'D', 3), ( 'E', 6)],
    'C': [( 'F', 5)],
    'D': [( 'G', 2)],
    'E': [( 'G', 1)],
    'F': [( 'G', 2)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 2,
    'G': 0
}

def a_star(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
```

```

came_from = {}
g_cost = {start: 0}

while not pq.empty():
    _, current = pq.get()

    if current == goal:
        path = []
        while current in came_from:
            path.append(current)
            current = came_from[current]
        path.append(start)
        path.reverse()

        print("Shortest Path:", " -> ".join(path))
        print("Total Cost:", g_cost[goal])
        return

    for neighbor, cost in graph[current]:
        new_cost = g_cost[current] + cost

        if neighbor not in g_cost or new_cost < g_cost[neighbor]:
            g_cost[neighbor] = new_cost
            f_cost = new_cost + heuristic[neighbor]
            pq.put((f_cost, neighbor))
            came_from[neighbor] = current

print("No Path Found")

a_star('A', 'G')

```

Output

```

Shortest Path: A -> B -> D -> G
Total Cost: 6

```

Result

The A* Search Algorithm was successfully implemented using Python. The algorithm efficiently found the shortest path from the start node (A) to the goal node (G) by combining the actual path cost $g(n)$ and the heuristic estimate $h(n)$. The shortest path obtained was:

$$A \rightarrow B \rightarrow D \rightarrow G$$

Conclusion

The A* Search Algorithm combines the strengths of Dijkstra's algorithm and greedy best-first search by using both the actual path cost $g(n)$ and a heuristic estimate $h(n)$ to guide the search. Since the heuristic used in this experiment is admissible (it never overestimates the true cost to the goal), A* guaranteed an optimal path from A to G while avoiding unnecessary exploration of costlier branches such as $A \rightarrow C \rightarrow F \rightarrow G$. This makes A* significantly more efficient than uninformed search methods for large weighted graphs, provided a good heuristic function is available.